

INF8460 – Traitement automatique de la langue naturelle

Semantic Textual Similarity

Rémy Dumas Mathias Gonin Waël Hamdan

Polytechnique Montréal

{remy.dumas, mathias.gonin, wael.hamdan}@polymtl.ca

Abstract

Notre travail s'est porté sur l'analyse de similarité entre des paires de phrases (tâche **STS**). L'objectif était d'utiliser un large éventail de techniques pour obtenir la représentation des phrases et les comparer entre elles, allant des plus basiques aux plus récentes. Nos meilleurs résultats ont été obtenus en utilisant le modèle XLNet, amenant à une corrélation de Spearman de 0.909. En comparaison, l'état de l'art se situe actuellement à 0.925 avec le modèle MTN-DNN-SMART de Microsoft (source: Glue Benchmark).

1 Introduction

La tâche STS consiste à déterminer la similarité de deux textes, en leur assignant par exemple un score entre 0 (pas du tout similaire) et 5 (même sens). La complexité de la tâche réside dans la complexité de la langue elle-même. Des modèles basiques seraient par exemple tout à fait acceptables pour le cas suivants : "Trois personnes jouent aux échecs", "Deux personnes jouent aux échecs". En effet, les phrases sont proches syntaxiquement et sémantiquement. Ces modèles sont par contre impuissants pour capter avec précision la similarité entre "Un homme retire des toasts du sac", "Un individu récupère les tartines du sachet". Inversement, ils peuvent assigner une forte similarité entre des phrases de sens éloignés : "Je compte sur toi pour la monnaie", "Je compte la monnaie".

On se rend compte assez vite qu'une représentation globale de la phrase est nécessaire. On peut la voir en première approche comme un assemblage de mots. On parlera alors de modèles *Bag of Words* (**BoW**), qui constituent notre *baseline*. Cependant un même mot peut avoir des significations bien différentes dans des contextes différents, comme "Seulement lui peut résoudre ce problème" et "Il peut résoudre seulement ce

problème". Prendre les mots sans garder l'ordre de la phrase ne permet donc pas d'atteindre de très bons résultats. La prise en compte du contexte, de l'ordre des mots, des dépendances à long-terme, ... sont une multitude de défis que l'on cherche désormais à relever à partir de méthodes neuronales avec l'apprentissage machine. Les solutions développées actuellement pour la tâche STS sont pour la plupart basées sur le modèle **BERT** (l'architecture transformeur, la partie encodeur, (Devlin et al., 2018)). Ces modèles ont été pré-entraînés sur une énorme quantité de donnée, donnant de bon résultats sur des tâches variées en traitement du langage (STS, MRPC, QQP, MNLI...). Ils peuvent être adaptés et entraînés sur une tâche spécifique (fine tuning) pour donner des résultats approchant ceux de l'état de l'art. Dans cet article nous traitons donc des *baseline* les plus répandues, de méthodes neuronales pour les améliorer et enfin de la combinaison de modèles pour augmenter encore les performances.

2 Travaux connexes

Une autre approche développée par Google permet d'obtenir directement des représentations contextuelles de phrases. L'*Universal Sentence Encoder* est un modèle pré-entraîné qui est fourni sous deux versions : l'une entraînée à l'aide d'une architecture d'encodeur des Transformers et l'autre entraînée à l'aide d'un DAN (Deep Averaging Network). Ce modèle encode chaque phrase en un vecteur de grande dimension qui peut ensuite être utilisée pour différentes tâches de NLP. Il suffit alors de calculer la similarité cosinus entre les deux vecteurs obtenus pour chaque paire de phrases afin de déterminer le degré de similarité sémantique. Les scores obtenus en STS sont de 0,815.

Google est aussi à l'origine du modèle AL-

BERT basé sur le BERT (Lan et al., 2019). Ce modèle présente quelques différences avec le BERT, en particulier le pré-entraînement qui utilise une tâche de Sentence Order Prediction (SOP). Le résultat de la tâche est positif si les deux phrases sont dans le bon ordre, et négatif si leur ordre est inversé. Ce modèle permet d’atteindre des scores de 0,925.

3 Données

Nous disposons de 5703 couples de phrases pour l’ensemble d’entraînement et 1463 pour l’ensemble de test. L’ensemble d’entraînement est séparé (90 %/10%) pour certains modèles afin de créer un ensemble de validation nécessaire pour adapter nos hyper-paramètres et prévenir l’over-fitting.

3.1 Analyse des données

On propose d’analyser les longueurs des phrases (en nombre de *tokens* avant pré-processing) pour analyser statistiquement les deux ensembles. Les résultats de cette analyse sont répertoriés dans le tableau 1.

	Données d’entraînement	Données de test
Moyenne	11.07	12.77
Ecart-type	6.66	6.23
min	3	3
25%	7	8
50%	9	11
75%	13	17
max	295	90

Table 1: Analyse de la longueur des phrases (nombre de tokens avant pré-processing) dans les corpus d’entraînement et de test

Les deux ensembles sont homogènes, à l’exception éventuellement de quelques phrases très longues dans le corpus d’entraînement. Il est alors justifié de se servir de l’ensemble d’entraînement et de l’ensemble de test conjointement.

3.2 Scores de référence

Un score de similarité entre 0 et 5 a été attribué à tous les couples de l’ensemble d’entraînement, en moyennant les scores d’un groupe d’humains. On observe un léger déséquilibre de la répartition en faveur des scores plus élevés. En conséquence, il

est possible que certains scores soient légèrement sur-évalués sur notre corpus de test.

4 Structure générale

Chacun de nos modèles a respecté une structure identique. La première étape est la transformation des données jusqu’à obtention des descripteurs des phrases. Ensuite vient la mesure de similarité grâce à un estimateur, et enfin la comparaison avec les données d’entraînement par la corrélation Spearman. L’ensemble de ces étapes peut être visualisé dans la figure 1.

4.1 Création des descripteurs

Un descripteur doit être un élément caractérisant au mieux une phrase. Ils doivent être suffisamment spécifiques pour que chaque phrase soit représentée de manière unique, et suffisamment simples pour pouvoir évaluer la similarité entre deux phrases. Dans cet article nous n’avons traité que des descripteurs sous forme de vecteurs pour pouvoir se placer dans un espace normalisé. Pour obtenir de tels vecteurs, les données initiales passent successivement à travers des ”transformeurs”. Parmi les plus utilisés, nous trouvons les transformeurs de pré-processing, ceux permettant de transformer les mots en vecteurs ou encore ceux permettant de rassembler des séquences de vecteurs en un vecteur unique représentatif de la phrase. Lors de la phase de pré-processing, nous avons généralement tokenisé les phrases, supprimé les caractères spéciaux et les stopwords, passé le texte en minuscules ou encore remplacé les mots par leur lemme ou par un synonyme. Ponctuellement nous avons utilisé d’autres transformeurs pour obtenir des étiquetages morpho-syntactiques et des distances d’édition minimale entre deux phrases.

4.2 Estimation de la similarité

Une fois que chaque phrase possède son descripteur nous pouvons évaluer leurs similarités. Plusieurs mesures peuvent être utilisées, dont la distance cosinus (1), la distance Manhattan (2) ou la distance euclidienne (3). La distance euclidienne est généralement déconseillée lorsque les descripteurs sont de dimension trop élevée. La distance Manhattan est largement utilisée notamment avec l’utilisation de modèle avec LSTM Siamois. Cependant nous avons obtenus de meilleurs résultats avec la distance cosinus.

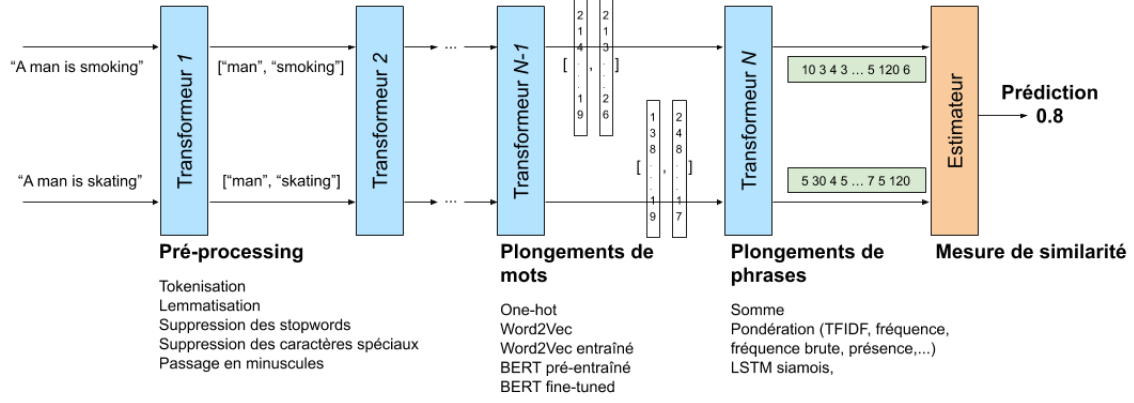


Figure 1: Pipeline générale : Application de transformeurs successifs jusqu'à obtention des descripteurs finaux avant évaluation de la similarité

$$d_{cos} = \frac{u_1 \cdot u_2}{||u_1|| ||u_2||} \quad (1)$$

$$d_{manh} = ||u_1 - u_2||_1 \quad (2)$$

$$d_{eucl} = ||u_1 - u_2||_2 \quad (3)$$

Il est également possible d'utiliser des perceptrons et perceptrons multi-couches pour obtenir un score de similarité. Pour cela, les deux vecteurs correspondants à une paire de phrases doivent d'abord être rassemblés, nous les avons concaténés. D'autres méthodes utilisent la sommation ou encore la moyenne entre les vecteurs des deux phrases. La dernière couche doit être composée d'un neurone unique.

4.3 Comparaison aux scores réels

Une fois tous les scores calculés et classés on peut les comparer aux *labels* (3.2). Deux corrélations sont classiquement appliquées : la corrélation de Pearson et la corrélation de Spearman (4) qui sont très similaires. Dans la corrélation de Spearman r_s , pour un échantillon de taille n , les variables de rang $cov(r_{g_x}, r_{g_y})$ sont calculées à partir des données X_i, Y_i .

$$r_s = \frac{cov(r_{g_x}, r_{g_y})}{\sigma_{r_{g_x}} \sigma_{r_{g_y}}} \quad (4)$$

avec:

$cov(r_{g_x}, r_{g_y})$ la covariance de variables de rang

$\sigma_{r_{g_x}} \sigma_{r_{g_y}}$ les écarts-type des variables de rang

Notre méthode d'évaluation est basée sur la corrélation de Spearman, qui a pour spécificité

d'examiner s'il existe une relation entre le rang des observations au lieu d'une relation entre les observations directement. Les sorties de nos modèles ne sont donc pas nécessairement entre 0 et 5, puisque seul leurs rangs est important. Cette corrélation nous donne un résultat compris entre -1 et 1, 1 signifiant que nos scores sont parfaitement corrélés avec les vrais scores de l'ensemble de test.

5 Méthodes utilisées

On suppose que l'étape de pré-processing a déjà été effectuée.

5.1 Bag-of-Words

Nous obtenons un vocabulaire de V mots après pré-processing. En associant à chaque type du vocabulaire un indice i , on leur associe un vecteur de dimension $|V|$ dont toutes les composantes sont nulles sauf la i -ème composante qui vaut 1. On parle alors de vecteurs *One-hot*. À partir d'une séquence de vecteurs *One-hot* (1 pour chaque token), on obtient par sommation une représentation de la phrase. Puis nous appliquons la pondération *TF-IDF*, qui permet d'évaluer l'importance d'un terme de la phrase, relativement aux autres phrases du corpus. Cette pondération permet d'augmenter significativement la performance du modèle, la corrélation de Spearman passe de 0.73 à 0.76. Les vecteurs de phrases ainsi obtenus sont ensuite comparés avec la similarité cosinus (1). Ce modèle est un modèle *Bag-of-Words*, l'information de contexte et de position est perdue, mais les performances sont bonnes relativement à la simplicité des techniques utilisées.

5.2 Plongements lexicaux *Word2Vec*

Les vecteurs *One-hot* présentent deux inconvénients majeurs. Ils sont de grande dimension si la taille du vocabulaire est importante (ce qui est peut être résolu par exemple par décomposition en valeurs singulières - *SVD*), c'est un inconvénient en terme de calcul si on veut les utiliser dans un algorithme par la suite, et ils ne permettent pas de prendre en compte les relations entre les mots.

On utilise alors des plongements lexicaux. Les plongements *Word2Vec* arrivent à représenter les tokens par des vecteurs denses et courts en entraînant un réseau de neurones à reconstruire le contexte linguistique des mots (tâche *skip-grams* par exemple). Ces plongements sont basés sur l'hypothèse distributionnelle, des mots apparaissant dans des contextes similaires auront une signification similaire. Nous avons récupérés des plongements pré-entraînés (Gensim, entraînés sur un corpus de 600 milliards de tokens) de dimension 300. Les vecteurs sont alors beaucoup plus courts (avantage pour le calcul dans un algorithme), et les vecteurs capturent des relations entre les mots. Par exemple, les mots "vacances" et "soleil" auront des plongements proches (avec la similarité cosinus).

Nous les avons simplement sommés pour obtenir des représentations de phrases. La comparaison se fait à nouveau par similarité cosinus (1).

5.3 Prise en compte de l'ordre - *LSTM*

Dans les méthodes précédentes on obtenait une représentation de la phrase par sommation des plongements lexicaux. Cependant cette technique ne permet pas de prendre en compte l'ordre et les dépendances des mots dans la phrase. Nous nous sommes tournés vers les modèles *LSTM* ayant la particularité de pouvoir traiter des problèmes séquentiels de longueur variable. Les deux phrases que l'on cherche à comparer ont d'abord été transformées en deux séquences de vecteurs (un vecteur par mot), qui passeront chacun dans un *LSTM* commun pour obtenir finalement deux représentations de phrases. On parle alors de *LSTM* siamois (Mueller and Thyagarajan, 2016). Les similarités cosinus (1) et Manhattan (2) ont été utilisées en sortie.

5.4 Architecture *BERT*: *BERT* et *XLNet*

Des meilleurs résultats ont été obtenus en utilisant l'architecture transformeur (partie encodeur) du *BERT*. Le modèle est d'abord entraîné sur deux tâches: prédire un mot masqué dans une phrase puis prédire si une phrase est la suite logique de la première phrase. Nous avons récupéré un modèle pré-entraîné (transfer-learning) sur une énorme quantité de données (Wikipédia Anglais et un corpus de livres): *BERT-large-uncased*. Nous avons ensuite adapté le modèle à notre tâche spécifique *STS* en lui rajoutant une couche de régression en sortie (fine-tuning). Le modèle prend en entrée les deux phrases concaténées et retourne un score. En plus des avantages du vecteur dense capturant les relations entre les mots, *BERT* permet d'obtenir des plongements contextuels (capture le contexte bidirectionnel), c'est à dire qu'un mot aura un plongement différent selon la phrase dans laquelle il se trouve.

Le modèle *XLNet* (Yang et al., 2019) obtient encore de meilleurs résultats. La phase de pré-entraînement diffère de celle de *BERT* avec un nouvel objectif de modélisation du langage par permutation. *XLNet* utilise les permutations des mots de la phrase pour prédire un autre mot et ainsi apprendre le contexte de manière bidirectionnelle.

5.5 Combinaison de modèles

Comme dit précédemment (3.2), les scores de référence ont été obtenus en moyennant des scores donnés par des humains. En effet, tout modèle, y compris le "modèle" humain, peut être considéré comme imparfait au vu de la complexité de la tâche. C'est le cas de tous nos modèles obtenus jusqu'à maintenant. On peut de manière analogue espérer tirer un "sens commun" en pondérant leur sortie (voir figure 2). Le modèle en charge de cette pondération est appelé méta-régresseur. Un ensemble de validation (15% de l'ensemble d'entraînement) est nécessaire pour déterminer les paramètres du méta-régresseur. Ce dernier favorisera nos meilleurs modèles. Une analyse de leur influence sur la prédiction finale a été faite en 6.2.2. Le méta-régresseur était soit une simple régression linéaire, soit un perceptron à une ou plusieurs couches.

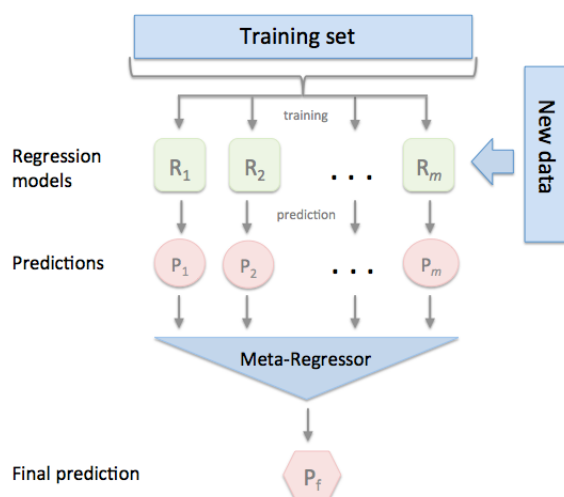


Figure 2: Principe de la méta-régression : Chaque modèle entraîné produit indépendamment des autres des scores de similarité sur l'ensemble de validation. Le méta-régresseur prend ces valeurs en entrée pour donner sa propre prédiction. Source: rasbt.github.io

6 Résultats

6.1 Modèles individuels

Le modèle individuel donnant les meilleurs résultats est le XLNet-Large avec une corrélation Spearman de 0.91 sur l'ensemble de test. Les autres scores sont répertoriés dans le tableau 2.

6.2 Méta-régression

6.2.1 Scores

Les modèles utilisés sont des *Bag-of-Words* utilisant diverses pondérations (décomptes et *TFIDF*) appliqués pour divers pré-processing (étiquetage morpho-syntaxique, lemmes, synonymes), des modèles utilisant les *embeddings* Word2Vec (avec et sans LSTM), et des modèles basés sur l'architecture BERT. Il faut ajouter une prédiction utilisant la distance d'édition minimale. Une régression linéaire a été appliquée pour combiner ces modèles, donnant une **corrélation Spearman de 0.905**. C'est moins élevé que le score de notre meilleur modèle XLNet-large, probablement à cause d'une influence trop importante des "mauvais" modèles.

6.2.2 Explicabilité

Nous avons utilisé la librairie SHAP (Lundberg and Lee, 2017) pour ajouter de l'explicabilité dans les résultats précédents. La librairie nous permet d'évaluer l'influence de chacun des modèles sur la prédiction finale. Comme on peut le voir sur la fig-

Plongements de mots	Plongements de phrases	Estimateur	Score
BERT-base pré-entraîné	Somme	Similarité Cosinus	0.41
One-hot	TF-IDF + SVD	Multi-Layer Perceptron	0.44
One-hot	TF-IDF	Multi-Layer Perceptron	0.44
x	Universal sentence encoder	Similarité Cosinus	0.73
Word2Vec (avec stopwords)	Somme	Similarité Cosinus	0.73
One-hot	Fréquence	Similarité cosinus	0.73
Word2Vec	LSTM	Similarité Cosinus	0.74
One-hot	TF-IDF	Similarité Cosinus	0.76
Word2Vec	Somme	Similarité Cosinus	0.78
x	x	BERT-base fine-tuned	0.87
x	x	BERT-large fine-tuned	0.89
x	x	XLNet-large fine-tuned	0.91

Table 2: Résultats

ure 3, les modèles basés sur l'architecture BERT sont considérés comme beaucoup plus importants.

7 Conclusion

Les meilleurs modèles sont ceux qui utilisent le transfer-learning, ayant été entraînés sur des masses de données très importantes. Après une étape de fine-tuning, les plongements contextuels obtenus permettent d'obtenir des résultats proches de ceux de l'état de l'art, en particulier pour le modèle XLNet. Ce dernier pourrait donner de meilleurs résultats avec des hyper-paramètres encore plus adaptés. Toutefois, on remarque que des modèles très basiques comme le bag-of-words sans pondération permettent aussi d'obtenir des résultats plutôt bons.

Les meilleurs modèles actuels pourraient sûrement être améliorés en captant les expressions imagées, le second degré ou encore l'humour, en étant entraînés sur des ensembles de phrases

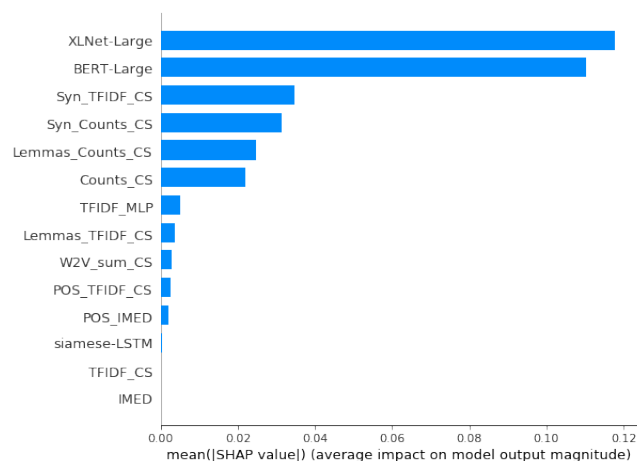


Figure 3: Évaluation de l’influence de chacun des modèles à partir des valeurs ”shap”

adéquats.

References

- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#). *arXiv e-prints*, page arXiv:1810.04805.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. [ALBERT: A Lite BERT for Self-supervised Learning of Language Representations](#). *arXiv e-prints*, page arXiv:1909.11942.
- Scott Lundberg and Su-In Lee. 2017. [A Unified Approach to Interpreting Model Predictions](#). *arXiv e-prints*, page arXiv:1705.07874.
- Jonas Mueller and Aditya Thyagarajan. 2016. [Siamese recurrent architectures for learning sentence similarity](#). In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, AAAI’16, pages 2786–2792. AAAI Press.
- Zhilin Yang, Zihang Dai, Yiming Yang, Jaime G. Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. 2019. [Xlnet: Generalized autoregressive pretraining for language understanding](#). *CoRR*, abs/1906.08237.